

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING



BIG PROJECT 2 REPORT

FPGA Implementation of 8-Point FFT - Digital System Design

SUPERVISOR: Nguyễn Trung Hiếu

SUBJECT: Digital System Design
and Verification

GROUP: 08

List of Members

STT	MSSV	Họ Và Tên	Lớp
1	2210780	Nguyễn Đại Đồng	L01
2	2213874	Nguyễn Thanh Tùng	L01
3	2213496	Nguyễn Quốc Tín	L01

Ho Chi Minh, ../../20..

Mục lục

1	Theoretical Background	1
1.1	Biến đổi Fourier Rời rạc (DFT)	1
1.2	Thuật toán FFT (Cooley-Tukey Radix-2)	1
1.3	Kiến trúc Tính toán Cơ sở	2
1.3.1	Đơn vị Cánh bướm (Butterfly Unit)	2
1.3.2	Đảo Bit (Bit Reversal)	2
1.4	Vấn đề Thực thi Phần cứng với Floating-Point	3
2	RTL Implementation Strategy	4
2.1	IEEE 754 Single-Precision Floating-Point Adder/Subtractor Design	4
2.2	IEEE 754 Single-Precision Floating-Point Multiplier Design	6
2.3	8-Point FFT Processor Top-Level Architecture	7
2.3.1	Thiết kế bộ FFT nguyên bản	7
2.3.2	Thiết kế bộ FFT sau khi tối ưu hóa	9
3	Design Verification and Evaluation	14
3.1	Verification for Floating-Point Adder/Subtractor Design	14
3.2	Verification for Floating-Point Multiplier Design	15
3.3	Verification for Radix-2 Butterfly Processing Unit (BPU) Design	15
3.4	Verification for 8-Point FFT Processor	15
4	Synthesis of FFT-8Points Design	16

Danh sách hình vẽ

1.1	Sơ đồ cấu trúc Butterfly Radix-2	2
2.1	Algorithmic flowchart of the IEEE 754 Floating-Point Adder/Subtractor.	4

2.2	Top-level hardware architecture of the 32-bit Floating-Point Adder/Subtractor. .	5
2.3	Algorithmic flowchart of the IEEE 754 Floating-Point Multiplier.	6
2.4	Top-level hardware architecture of the 32-bit Floating-Point Multiplier.	7
2.5	Structure of Radix-2 Butterfly Processing Unit (BPU).	8
2.6	Structure of 8-Point FFT Processor Original.	9
2.7	Bộ BFP ở tầng 1 của bộ FFT.	10
2.8	Bộ BFP ở tầng 2 của bộ FFT.	11
2.9	Bộ Complex Multiplier.	12
2.10	Bộ BFP ở tầng 3 của bộ FFT.	13
2.11	Thiết kế của bộ FFT-8Point sau khi tối ưu hóa.	13

Danh sách bảng

List of Listings

1	Kết quả test của bộ cộng trừ FPU.	14
2	Kết quả test của bộ nhân FPU.	15

Chapter 1. Theoretical Background

1.1 Biến đổi Fourier Rời rạc (DFT)

Trước khi thiết kế bộ FFT, cần xem xét định nghĩa gốc của Biến đổi Fourier Rời rạc (DFT). Với một tín hiệu đầu vào $x[n]$ có độ dài N , DFT được định nghĩa như sau:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}, \quad 0 \leq k \leq N-1 \quad (1.1)$$

Trong đó:

- $x[n]$: Chuỗi tín hiệu đầu vào miền thời gian.
- $X[k]$: Chuỗi tín hiệu đầu ra miền tần số.
- W_N : Hệ số quay (Twiddle Factor), được xác định bởi:

$$W_N = e^{-j\frac{2\pi}{N}} = \cos\left(\frac{2\pi}{N}\right) - j \sin\left(\frac{2\pi}{N}\right) \quad (1.2)$$

Hạn chế của DFT trực tiếp: Để tính toán trực tiếp, độ phức tạp thuật toán là $O(N^2)$. Với $N = 8$, số phép nhân phức cần thực hiện là $8^2 = 64$. Điều này tiêu tốn nhiều tài nguyên phần cứng và làm giảm tốc độ xử lý.

1.2 Thuật toán FFT (Cooley-Tukey Radix-2)

Để tối ưu hóa phần cứng, thuật toán *Cooley-Tukey Radix-2 Decimation-in-Time (DIT)* được sử dụng. Thuật toán này chia nhỏ bài toán DFT N điểm thành hai bài toán DFT $N/2$ điểm (chẵn và lẻ) một cách đệ quy.

Công thức tổng quát cho Radix-2 DIT:

$$X[k] = \text{DFT}_{N/2}\{x_{chan}\} + W_N^k \cdot \text{DFT}_{N/2}\{x_{le}\} \quad (1.3)$$

Với thuật toán này, độ phức tạp giảm xuống còn $O(N \log_2 N)$. Đối với $N = 8$, số tầng tính toán là $\log_2 8 = 3$ tầng, và tổng số phép tính cơ sở giảm đáng kể so với DFT truyền thống.

1.3 Kiến trúc Tính toán Cơ sở

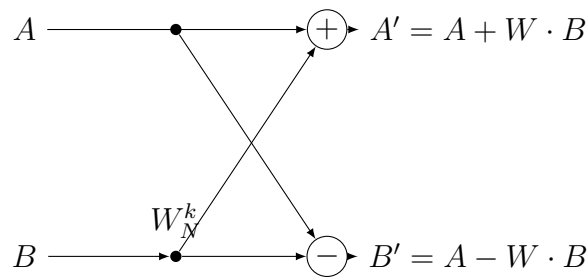
1.3.1 Đơn vị Cánh bướm (Butterfly Unit)

Phần tử xử lý trung tâm của FFT Radix-2 là cấu trúc Butterfly. Một đơn vị Butterfly thực hiện tính toán trên hai mẫu đầu vào A và B để tạo ra hai mẫu đầu ra A' và B' theo công thức:

$$A' = A + W_N^k \cdot B \quad (1.4)$$

$$B' = A - W_N^k \cdot B \quad (1.5)$$

Dưới đây là sơ đồ luồng tín hiệu của một đơn vị Butterfly:



Hình 1.1: Sơ đồ cấu trúc Butterfly Radix-2

Về mặt phần cứng, mỗi Butterfly đòi hỏi:

- 1 Bộ nhân phức (Complex Multiplier).
- 2 Bộ cộng/trừ phức (Complex Adder/Subtractor).

1.3.2 Đảo Bit (Bit Reversal)

Do tính chất phân chia "Decimation-in-Time", đầu vào của bộ FFT cần được sắp xếp lại theo thứ tự đảo bit để đầu ra có thứ tự tự nhiên. Ví dụ với $N = 8$ (biểu diễn bằng 3 bit):

- Index 1 (001_2) $\rightarrow$ Đổi thành 4 (100_2).
- Index 3 (011_2) $\rightarrow$ Đổi thành 6 (110_2).

Trong thiết kế phần cứng, việc này thường được thực hiện bằng cách trao đổi đường dây kết nối (hard-wired) ở tầng đầu vào, không tốn tài nguyên logic.

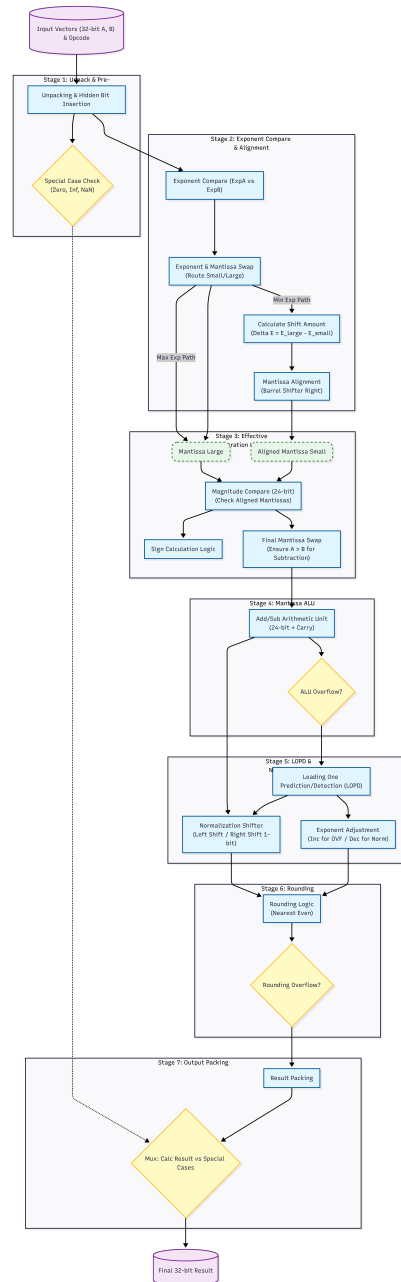
1.4 Vấn đề Thực thi Phần cứng với Floating-Point

Khác với Fixed-point, việc thiết kế FFT sử dụng Floating-point (thường theo chuẩn IEEE 754 Single Precision 32-bit) mang lại độ chính xác cao và dải động (dynamic range) lớn, nhưng đặt ra các thách thức thiết kế sau:

1. **Định dạng dữ liệu (IEEE 754 Standard):** Dữ liệu được biểu diễn dưới dạng $V = (-1)^S \times 1.M \times 2^{E-Bias}$. Mỗi số phức gồm 2 thanh ghi 32-bit (Phần thực và Phần ảo).
2. **Độ phức tạp của khối tính toán:**
 - *Phép cộng/trừ:* Cần các khối logic cho việc so sánh số mũ, dịch bit để căn chỉnh phần định trị (alignment), cộng phần định trị, và dò tìm bit 1 đầu tiên (Leading One Detector) để chuẩn hóa lại.
 - *Phép nhân:* Cần nhân phần định trị và cộng số mũ, đồng thời xử lý tràn số (overflow/underflow).
3. **Độ trễ đường ống (Pipeline Latency):** Do cấu trúc phức tạp của bộ cộng/nhân dấu phẩy động, mỗi phép tính thường mất nhiều chu kỳ xung nhịp (ví dụ: 3-5 cycles). Hệ thống cần thiết kế pipeline sâu để duy trì thông lượng (throughput) mà không làm giảm tần số hoạt động.
4. **Lưu trữ Hệ số quay:** Các giá trị W_N được lưu trữ trong ROM dưới định dạng IEEE 754 32-bit.

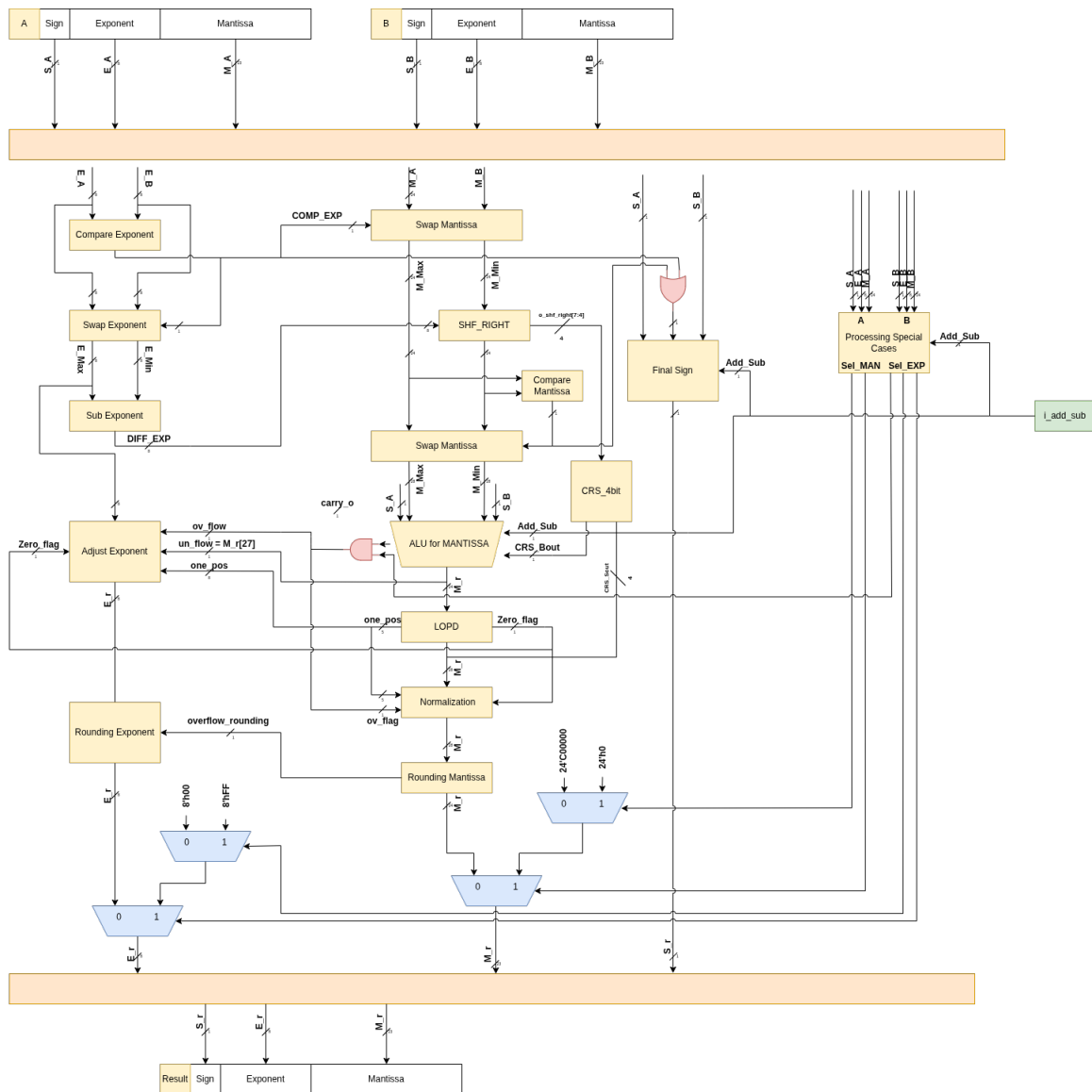
Chapter 2. RTL Implementation Strategy

2.1 IEEE 754 Single-Precision Floating-Point Adder/Subtractor Design



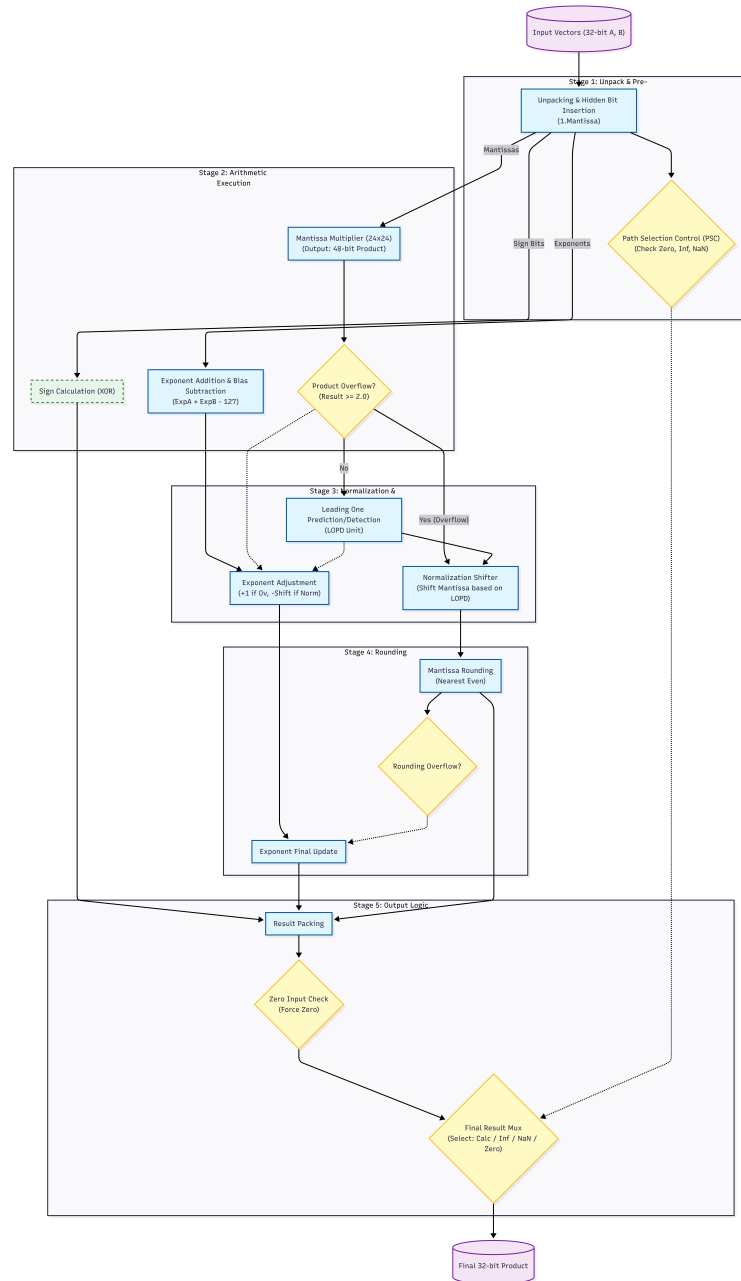
Hình 2.1: Algorithmic flowchart of the IEEE 754 Floating-Point Adder/Subtractor.

2 RTL IMPLEMENTATION STRATEGY

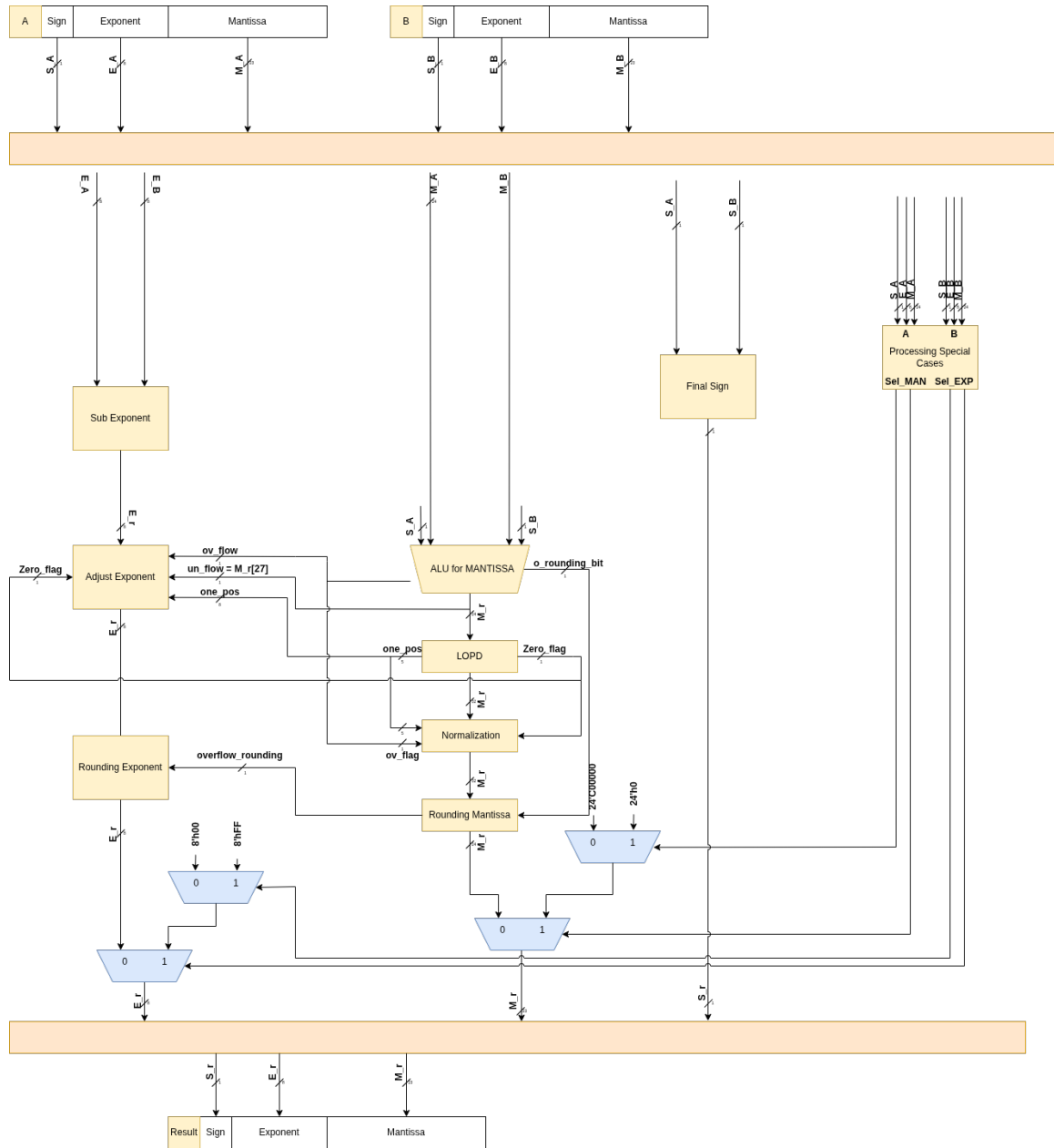


Hình 2.2: Top-level hardware architecture of the 32-bit Floating-Point Adder/Subtractor.

2.2 IEEE 754 Single-Precision Floating-Point Multiplier Design



Hình 2.3: Algorithmic flowchart of the IEEE 754 Floating-Point Multiplier.



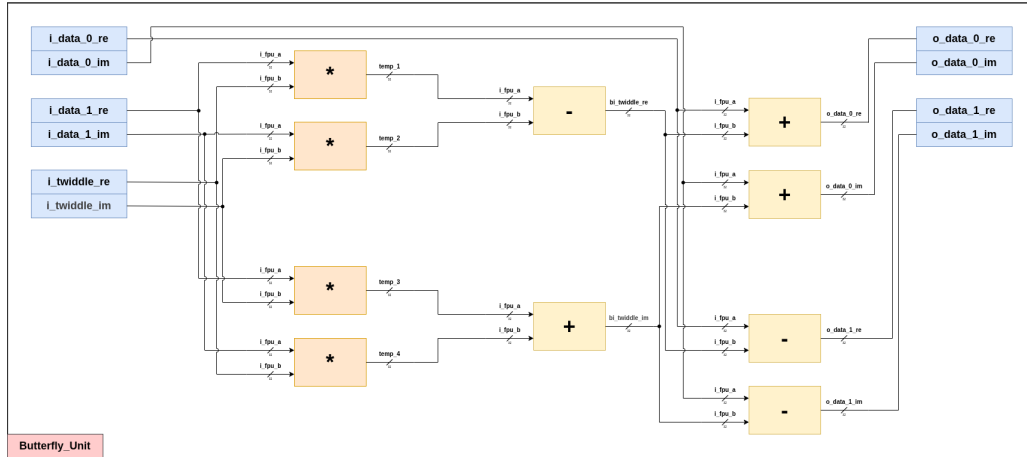
Hình 2.4: Top-level hardware architecture of the 32-bit Floating-Point Multiplier.

2.3 8-Point FFT Processor Top-Level Architecture

2.3.1 Thiết kế bộ FFT nguyên bản

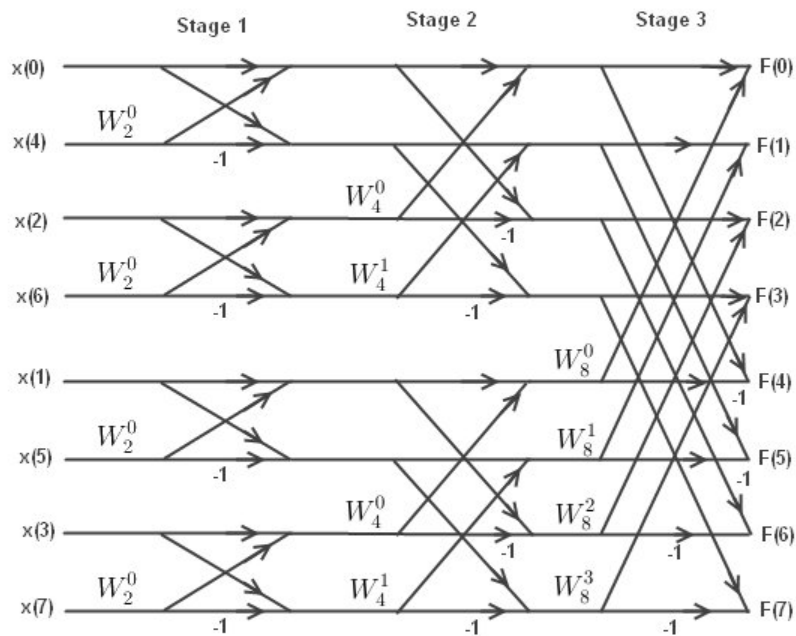
Đầu tiên, ta thiết kế một bộ Butterfly Processor Unit thực hiện phép nhân số phức như sau:

$$(a + jb) * (c + jd) = ac + jad + jbc - bd = (ac - bd) + j(ad + bc)$$



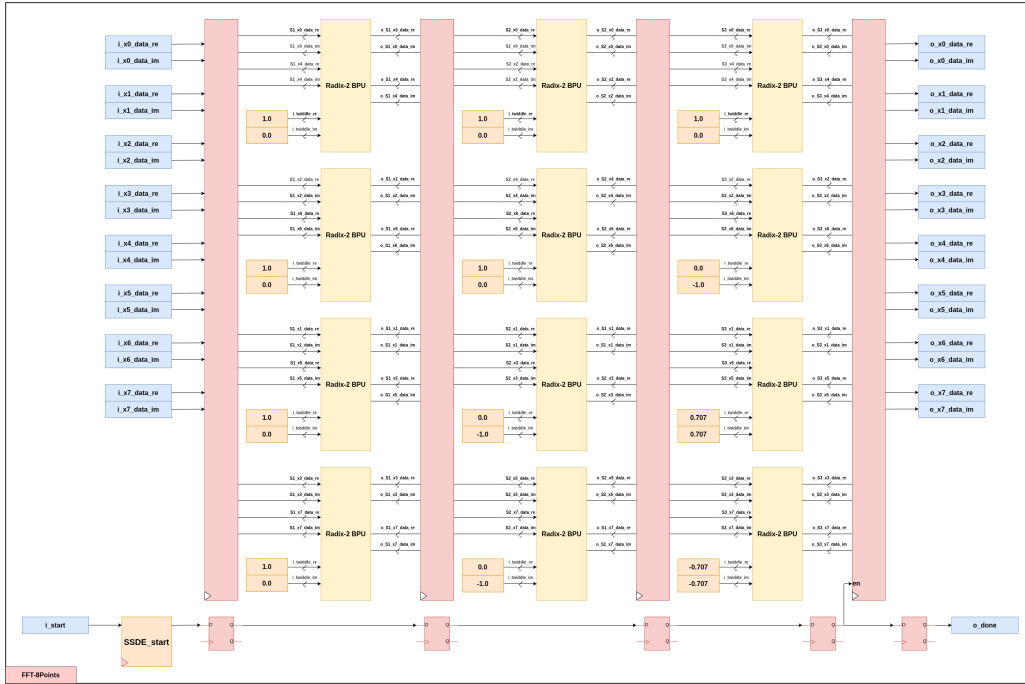
Hình 2.5: Structure of Radix-2 Butterfly Processing Unit (BPU).

Sử dụng bộ trên để thực hiện theo bộ tính FFT theo sơ đồ như sau:



Từ đó, ta có thiết kế RTL như sau:

2 RTL IMPLEMENTATION STRATEGY



Hình 2.6: Structure of 8-Point FFT Processor Original.

Tuy nhiên, với thiết kế 2.6 thì sử dụng đến 12 bộ Butterfly Processor Unit (BPU) trong thiết kế tổng, như vậy cần sử dụng đến:

- Bộ cộng/trừ: 72 bộ.
- Bộ nhân: 48 bộ.

Như vậy, với thiết kế nguyên bản sử dụng rất nhiều tài nguyên.

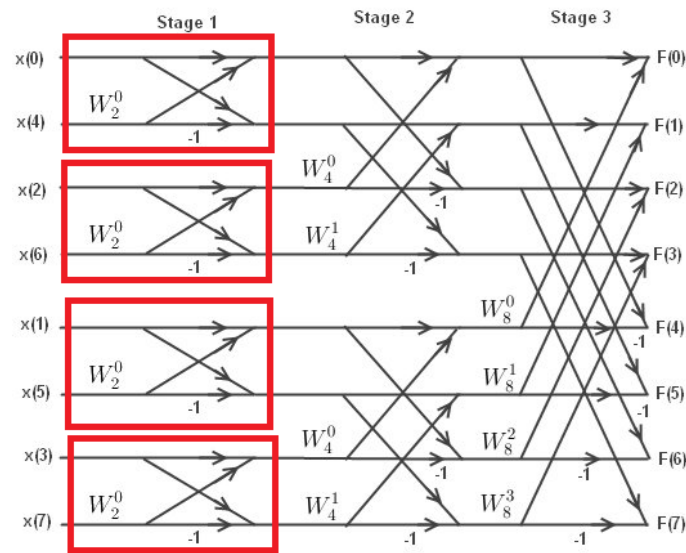
2.3.2 Thiết kế bộ FFT sau khi tối ưu hóa

Ta thấy khi sử dụng kiến trúc FFT 8-Point như hình 2.6, ta nhận thấy có các vị trí trong vòng tròn pha (Twiddle) có các giá trị như:

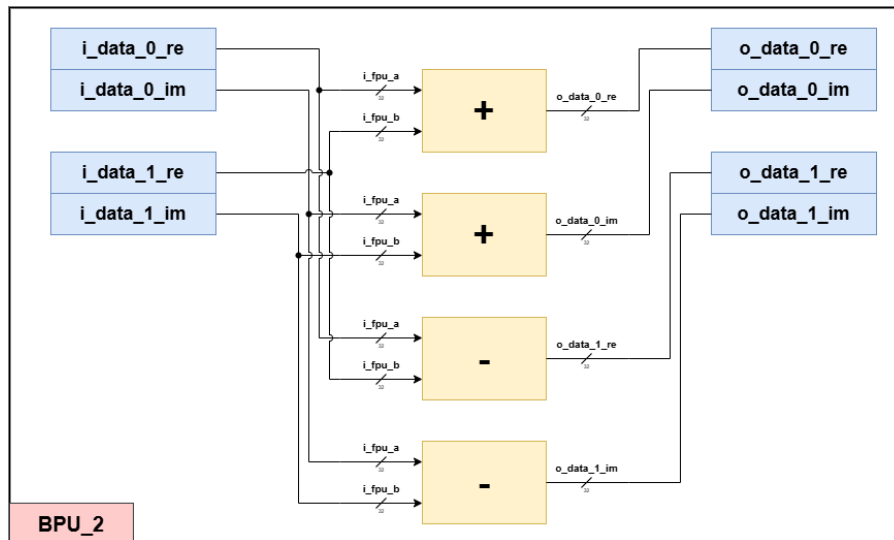
- $W_8^0 = 1$
- $W_8^1 = \cos(\frac{\pi}{4}) - j \sin(\frac{\pi}{4}) = \frac{\sqrt{2}}{2} - j \frac{\sqrt{2}}{2} \approx 0.7071 - j0.7071$
- $W_8^2 = -j$
- $W_8^3 = \cos(\frac{3\pi}{4}) - j \sin(\frac{3\pi}{4}) = -\frac{\sqrt{2}}{2} - j \frac{\sqrt{2}}{2} \approx -0.7071 - j0.7071$

2 RTL IMPLEMENTATION STRATEGY

1. Ở tầng 1:



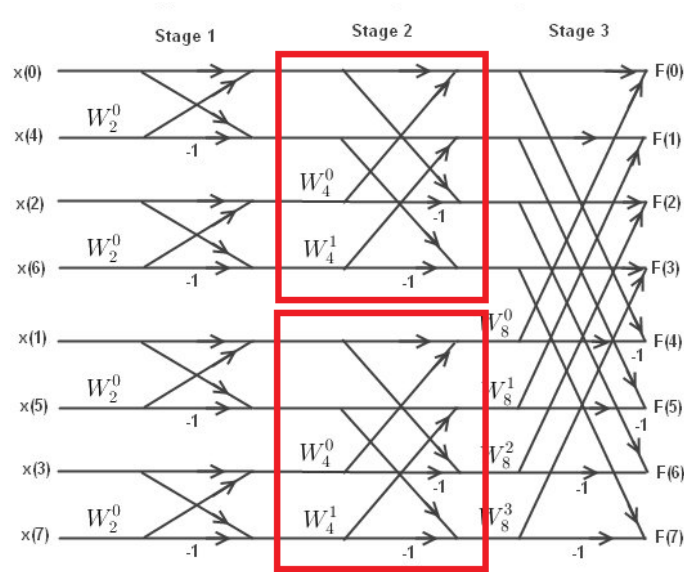
Các Butterfly chỉ nhân với giá trị $W_2^0 = 1$ nghĩa là không cần sử dụng bộ nhân trong thiết kế. Vì thế ta có thể thiết kế bộ BPU ở tầng 1 như sau:



Hình 2.7: Bộ BFP ở tầng 1 của bộ FFT.

2. Ở tầng 2:

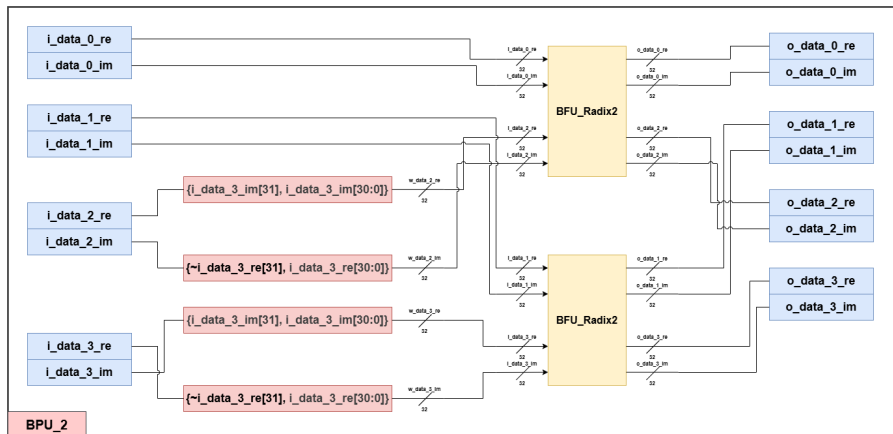
2 RTL IMPLEMENTATION STRATEGY



Các Butterfly cần xử lý hai giá trị của Twiddle là $W_4^0 = 1$ và $W_4^1 = -j$. Với giá trị $W_4^0 = 1$ thì ta thực hiện tương tự như bộ BPU ở tầng 1, còn giá trị $W_4^1 = -j$ thì ta nhận thấy khi:

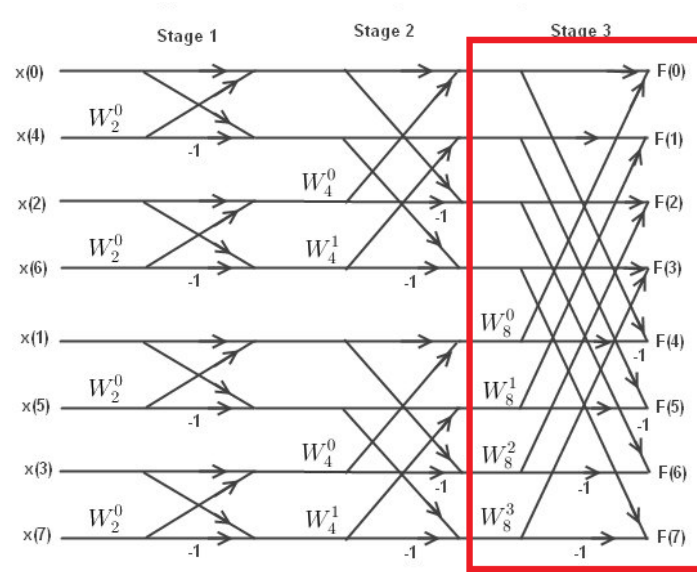
$$(a + jb) * (-j) = b - ja$$

Có nghĩa là giá trị tại $X(6)$ sẽ đảo giá trị vị trí phần thực và phần ảo với nhau và thực hiện bù dấu cho phần ảo.

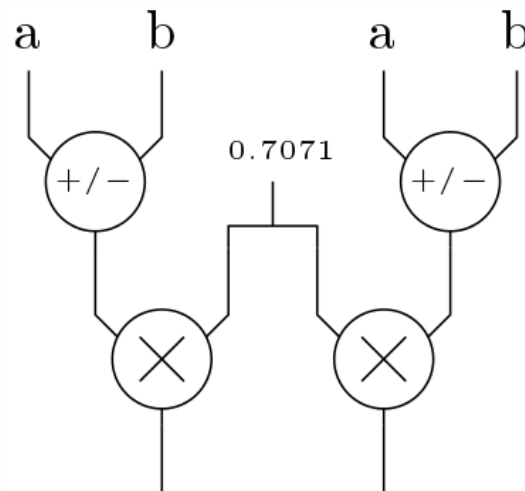


Hình 2.8: Bộ BFP ở tầng 2 của bộ FFT.

3. Ở tầng 3:



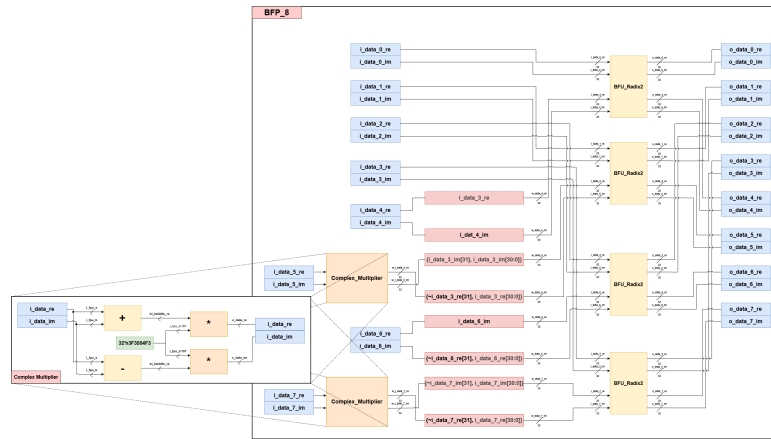
Các Butterfly cần xử lý hai giá trị của Twiddle là $W_8^0 = 1$ và $W_8^1 = 0.7071 - j0.7071$, $W_8^2 = -j$, $W_8^3 = -0.7071 - j0.7071$. Với giá trị $W_8^0 = 1$ và $W_8^2 = -j$ xử lý tương tự với ở tầng 2, các giá trị W_8^1 và W_8^3 ta cần thêm một bộ để xử lý phép toán nhân với giá trị $\sqrt{2}$:



Hình 2.9: Bộ Complex Multiplier.

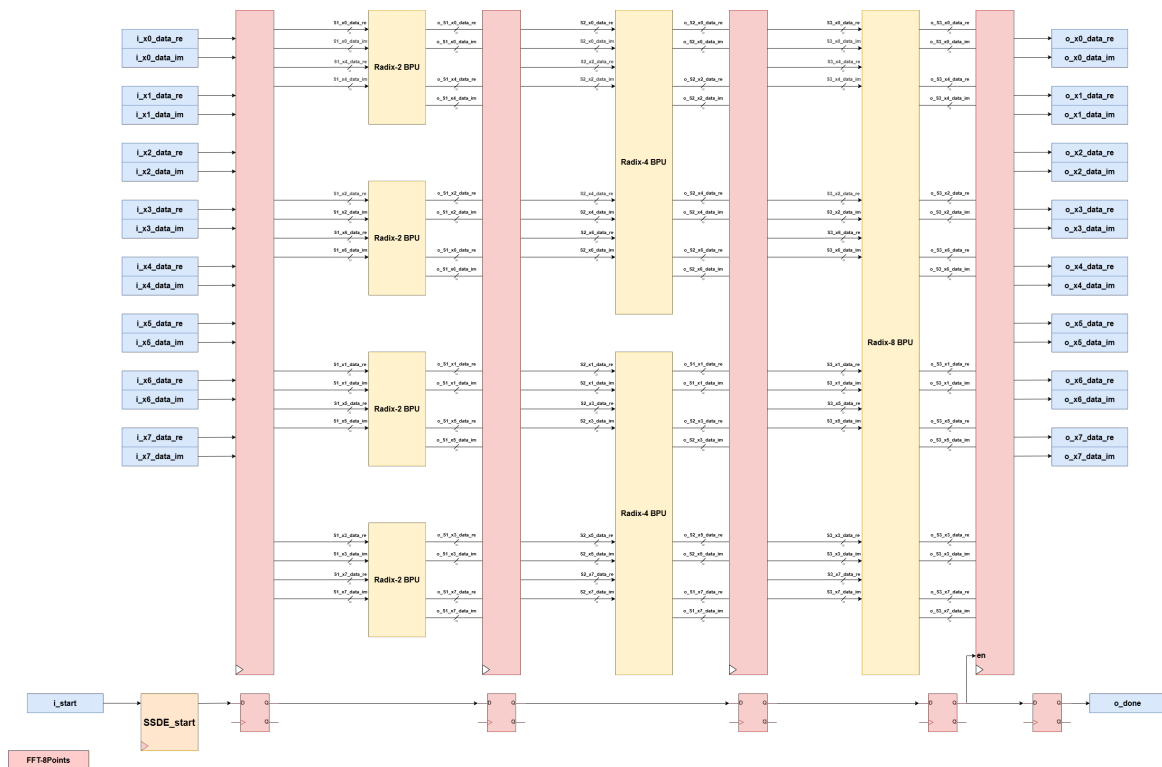
Như vậy tổng quan của bộ Butterfly ở tầng 3 như sau:

2 RTL IMPLEMENTATION STRATEGY



Hình 2.10: Bộ BFP ở tầng 3 của bộ FFT.

Từ các bộ Butterfly trên ta tổng hợp lại thành bộ FFT như sau:



Hình 2.11: Thiết kế của bộ FFT-8Point sau khi tối ưu hóa.

Với số lượng bộ cộng, trừ, nhân floating như sau:

- Bộ cộng/trừ: 44 bộ.
- Bộ nhân: 2 bộ.

Chapter 3. Design Verification and Evaluation

3.1 Verification for Floating-Point Adder/Subtractor Design

```

===== [ (0.0 & 0.0) ] =====
[ZERO][ADD]i_32_a=00000000 (0.000000000000000000000000) + i_32_b=00000000 (0.000000000000000000000000) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][ADD]i_32_a=00000000 (0.000000000000000000000000) + i_32_b=00000000 (0.000000000000000000000000) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=00000000 (0.000000000000000000000000) - i_32_b=00000000 (0.000000000000000000000000) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=00000000 (0.000000000000000000000000) - i_32_b=00000000 (0.000000000000000000000000) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
===== [ (0.0 & -0.0) ] =====
[ZERO][ADD]i_32_a=4016a197 (2.353612661361694335937500) + i_32_b=4016a197 (2.353612661361694335937500) | o_32_s=4096a197
(4.707225322723388671875000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=4.707225322723388671875000 (4096a197), dut=4.707225322723388671875000 (4096a197), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][ADD]i_32_a=4016a197 (2.353612661361694335937500) + i_32_b=4016a197 (2.353612661361694335937500) | o_32_s=4096a197
(4.707225322723388671875000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=4.707225322723388671875000 (4096a197), dut=4.707225322723388671875000 (4096a197), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=4016a197 (2.353612661361694335937500) - i_32_b=4016a197 (2.353612661361694335937500) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=4016a197 (2.353612661361694335937500) - i_32_b=4016a197 (2.353612661361694335937500) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
...
===== [ ADD_CANCEL_1 ] =====
FPU_A = 40a00000 (5.0000)
FPU_B = c0a00000 (-5.0000)
[ADD][ADD]i_32_a=40a00000 (5.000000000000000000000000) + i_32_b=c0a00000 (-5.000000000000000000000000) | o_32_s=00000000
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=0.000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ADD][ADD]i_32_a=c0a00000 (-5.000000000000000000000000) + i_32_b=40a00000 (5.000000000000000000000000) | o_32_s=80000000
(-0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (00000000), dut=-0.000000000000000000000000 (80000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
[ADD][SUB]i_32_a=40a00000 (5.000000000000000000000000) - i_32_b=c0a00000 (-5.000000000000000000000000) | o_32_s=41200000
(10.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=10.000000000000000000000000 (41200000), dut=10.000000000000000000000000 (41200000), rounding_error
=0.00000000 % (exp_error = 0.00001192 %)
[ADD][SUB]i_32_a=c0a00000 (-5.000000000000000000000000) - i_32_b=40a00000 (5.000000000000000000000000) | o_32_s=c1200000
(-10.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=-10.000000000000000000000000 (c1200000), dut=-10.000000000000000000000000 (c1200000), rounding_error
=0.00000000 % (exp_error = 0.00001192 %)

=====
===== TEST SUMMARY =====
Total test cases: 8252
Passed           : 8252
Failed           : 0
Pass rate        : 100.00%
=====

```

Listing 1: Kết quả test của bộ cộng trừ FPU.

3.2 Verification for Floating-Point Multiplier Design

```

===== [ W=1 : w.re * b.re ] =====
[MUL][MUL]i_32_a=3f800000 (1.00000000000000000000000000000000) * i_32_b=40400000 (3.00000000000000000000000000000000) | o_32_m=40400000
(3.00000000000000000000000000000000)
=> PASS: expect=3.00000000000000000000000000000000 (40400000), dut=3.00000000000000000000000000000000 (40400000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
===== [ W=1 : w.im * b.im ] =====
[MUL][MUL]i_32_a=00000000 (0.00000000000000000000000000000000) * i_32_b=40800000 (4.00000000000000000000000000000000) | o_32_m=00000000
(0.00000000000000000000000000000000)
=> PASS: expect=0.00000000000000000000000000000000 (00000000), dut=0.00000000000000000000000000000000 (00000000), rounding_error=0.00000000
% (exp_error = 0.00001192 %)
...
===== [ Large : w.im * b.im ] =====
[MUL][MUL]i_32_a=bf34fdf4 (-0.707000017166137695312500) * i_32_b=447a0000 (1000.000000000000000000000000000000) | o_32_m=
c430c001 (-707.000061035156250000000000000000)
=> PASS: expect=-707.000000000000000000000000000000 (c430c000), dut=-707.0000610351562500000000000000 (c430c001), rounding_error
=0.00000863 % (exp_error = 0.00001192 %)
===== [ Large : w.re * b.im ] =====
[MUL][MUL]i_32_a=3f34fdf4 (0.707000017166137695312500) * i_32_b=447a0000 (1000.000000000000000000000000000000) | o_32_m=4430c001
(707.000061035156250000000000000000)
=> PASS: expect=707.000000000000000000000000000000 (4430c000), dut=707.0000610351562500000000000000 (4430c001), rounding_error
=0.00000863 % (exp_error = 0.00001192 %)
===== [ Large : w.im * b.re ] =====
[MUL][MUL]i_32_a=bf34fdf4 (-0.707000017166137695312500) * i_32_b=c47a0000 (-1000.000000000000000000000000000000) | o_32_m=4430
c001 (707.000061035156250000000000000000)
=> PASS: expect=707.000000000000000000000000000000 (4430c000), dut=707.0000610351562500000000000000 (4430c001), rounding_error
=0.00000863 % (exp_error = 0.00001192 %)

=====
===== TEST SUMMARY =====
Total test cases:      36
Passed                :   36
Failed                :    0
Pass rate             : 100.00%
=====

```

Listing 2: Kết quả test của bộ nhân FPU.

3.3 Verification for Radix-2 Butterfly Processing Unit (BPU) Design

3.4 Verification for 8-Point FFT Processor

Chapter 4. Synthesis of FFT-8Points Design